Hand Gesture Controlled Car using ESP32-S3

Vashisth Chandak, *Task Phase Member, MRM,* Bhakti Haridad Rode, *Task Phase Member, MRM,*

***Abstract*—This project presents the design and implementation of a wireless, gesture-controlled vehicle utilizing the ESP32-S3 microcontroller. The system employs two MCU nodes: a wearable transmitter mounted on the user's hand and a receiver integrated into the robotic vehicle. By leveraging an MPU-6050 inertial measurement unit, the transmitter tracks hand orientation with precision, applying a complementary filter for smooth data processing. Communication is established via the ESP-NOW protocol, providing a connectionless, low-latency link critical for real-time tele-operation. Safety is prioritized through an integrated capacitive touch safety interlock on the wearable node and ultrasonic-based obstacle detection on the vehicle for collision avoidance. The dual-core architecture and hardware floating-point unit of the ESP32-S3 are fully utilized to eliminate input lag and ensure deterministic control, offering a more intuitive alternative to traditional joystick-based systems.**

***Index Terms*—Capacitive touch sensors, Complementary filter, Dual-core asymmetrical multiprocessing, ESP-NOW protocol, Gesture-based control, Hardware Floating Point Unit (FPU), Inertial Measurement Unit (IMU), Pulse Width Modulation (PWM)**

I. Introduction

In this project, we propose the design of a gesture-controlled vehicle. Where a person uses hand gestures to control a vehicle or a robot, it utilizes two ESP32-S3 MCU, one attached to the backside of the hand.i.e., the wrist, along with the MPU-6050 and a battery for their operation, while the other MCU is attached to the vehicle or the robot. The second MCU can identify the gestures made by the person and send them from the MCU attached to the wrist. It will read and identify the signals coming from the MCU and execute the desired movement or function accordingly. It will work like a remote controlled car.

It has many real life applications, such as steering an RC car, controlling a drone, or navigating robotic systems effortlessly. Unlike traditional control methods that rely on buttons, joysticks, or voice commands, gesture-based control offers a more intuitive and immersive experience. The accelerometer and gyroscope inside the MPU6050 track hand movements with precision. The ESP32-S3 then processes this data, applies a complementary filter for smoother motion tracking, and transmits the information wirelessly via Bluetooth or Wi-Fi to the controlled device. [1]

We are also using an ultrasonic sensor for obstacle detection therefore avoiding crashes. The system utilizes the integrated capacitive touch sensors of the ESP32-S3 to implement a safety interlock. Conductive copper tape is applied to the thumb and index finger regions of the wearable glove. By connecting the thumb pad to a designated Touch GPIO and the index pad to the common ground (GND), a 'pinch' gesture completes the circuit. This hardware-software integration ensures that the vehicle only responds to orientation data when the user provides an active enable signal, effectively filtering out non-intentional hand movements.

II. Why ESP32-S3 is better than other microcontrollers for this project

1) Dual-Core Asymmetrical Multiprocessing: The ESP32-S3 contains two Xtensa 32-bit LX7 CPU cores. In most microcontrollers like Arduino UNO run "Super-Loop" code where everything happens sequentially on one core. If the Wi-Fi stack needs to process data, it pauses the sensor reading, causing "jitter" in the vehicle's or robot's movement. We implement Task Pinning using FreeRTOS. Core 1 is dedicated to the high-speed I2C sampling of the MPU-6050 and the execution of the Complementary Filter. Core 0 is dedicated to the ESP-NOW radio stack.
2) Hardware Floating Point Unit (FPU): Calculating 3D orientation requires trigonometry $(\sin, \cos, \tan)$ and square roots. On an Arduino Uno (8-bit), these are handled via "Software Emulation," which is thousands of times slower than hardware-level math. The ESP32-S3 has a dedicated Single-Precision Hardware FPU. We can run complex Sensor Fusion algorithms (like the Complementary Filter) at a high frequency (e.g., 200Hz). This eliminates "input lag." When you tilt your hand, the FPU calculates the new angle instantly.
3) ESP-NOW: ESP-NOW is a connectionless protocol that operates at the MAC layer (Data Link Layer). It allows the Glove to "blast" small packets of data directly to the Car's or robot's MAC address. We achieve a latency of ¡10ms. For a moving vehicle, this is a safety requirement. If the user tilts their hand to stop, the car must receive that command immediately to avoid a collision.
4) Integrated Capacitive Touch Peripherals: The ESP32-S3 has 14 internal channels that measure changes in electrical capacitance. Unlike other MCUs, it doesn't need external pull-up resistors or touch ICs. It creates a Safety Interlock. The vehicle only moves when the user "pinches" their fingers. This removes the need for bulky mechanical buttons on the wearable glove, making it lighter.
5) Deep Sleep and ULP Co-Processor: Power consumption is the biggest challenge for wearables. Running the S3 at 240MHz will drain a small battery in hours. The S3 can enter Deep Sleep, where only a tiny "Ultra-Low-Power" (ULP) co-processor stays awake to monitor the Touch Pins. The glove can stay in a "Standby" state for weeks. The moment the user performs the "Pinch" gesture, the ULP senses the touch and triggers a Hardware Wake-

Up of the main cores. This creates a ”Seamless On” experience without needing a physical power switch.

III. Hardware Components Required for this Project

A. The Transmitter Node (The Gesture Glove)

1) ESP32-S3: The ”Brain” of the transmitter. It samples the raw data from the MPU-6050 via I2C, executes the Complementary Filter to determine the hand’s tilt angle, and packages that data into a small payload to be sent via ESP-NOW. The S3 is used specifically for its Hardware FPU (to handle tilt math) and its Dual-Core architecture (to ensure the sensor-reading doesn’t lag during transmission).

2) MPU-6050: Orientation Sensor. It contains a 3-axis accelerometer and a 3-axis gyroscope. It measures the force of gravity (to find ”Down”) and the angular velocity (to find ”Rotation”). When mounted on the back of the hand, it provides a full 360° map of the user’s wrist movement.

The MPU6050 is a very popular accelerometer gyroscope sensor chip that has six-axis sensing with a 16-bit measurement resolution The MPU6050 module has a total of 8 pins as shown in the figure 1.

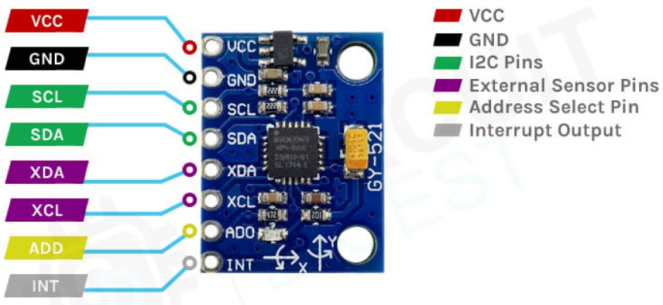

Fig. 1. Pinout Diagram of MPU-6050.

- VCC: provides power for the module.
- GND: Ground is connected to the Ground pin.
- SCL: Serial Clock Used for providing a clock pulse for I2C Communication.
- SDA: Serial Data Used for transferring Data through I2C communication.
- XDA: Auxiliary Serial Data - Can be used to interface other I2C modules with MPU6050.
- XCL: Auxiliary Serial Clock - Can be used to interface other I2C modules with MPU6050.
- ADD/ADO: Address select pin if multiple MPU6050 modules are used.
- INT: Interrupt pin to indicate that data is available for the MCU to read.

The MPU6050 module consists of an MPU6050 IMU chip from TDK InvenSense. It comes in a 24-pin QFN package with dimensions of 4mm x 4mm x 0.9mm. The module has a very low component count, including an AP2112K 3.3V regulator, I2C pull-up resistors, and bypass capacitors. There is also a power LED that indicates the power status of the module.

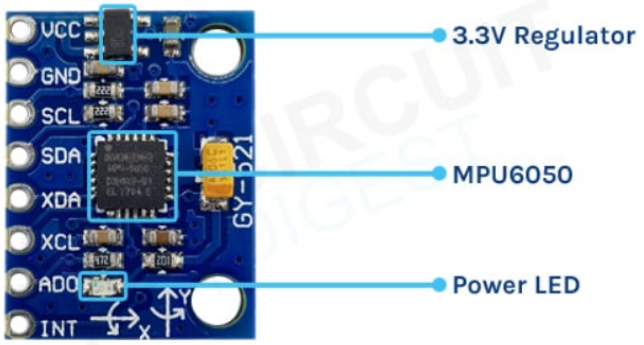

Fig. 2. Parts of MPU-6050.

The MPU6050 is a Micro-Electro-Mechanical Systems (MEMS) device that consists of a 3-axis Accelerometer and a 3-axis Gyroscope. This helps us to measure acceleration, velocity, orientation, displacement, and many other motion-related parameters of a system or object. This module also has a (DMP) Digital Motion Processor inside it, which is powerful enough to perform complex calculations and thus free up the work for the microcontroller.

MEMS Accelerometer: MEMS accelerometers are used wherever there is a need to measure linear motion, either movement, shock, or vibration, but without a fixed reference. They measure the linear acceleration of whatever they are attached to. All accelerometers work on the principle of a mass on a spring. When the thing they are attached to accelerates, then the mass wants to remain stationary due to its inertia, and therefore the spring is stretched or compressed, creating a force which is detected and corresponds to the applied acceleration.

MEMS Gyroscope: The MEMS Gyroscope works based on the Coriolis Effect. The Coriolis Effect states that when a mass moves in a particular direction with velocity and an external angular motion is applied to it, a force is generated and which causes a perpendicular displacement of the mass. The force that is generated is called the Coriolis Force, and this phenomenon is known as the Coriolis Effect. The rate of displacement will be directly related to the angular motion applied. The MEMS Gyroscope contains a set of four proof masses and is kept in a continuous oscillating movement. When an angular motion is applied, the Coriolis Effect causes a change in capacitance between the masses depending on the axis of the angular movement. This change in capacitance is sensed and then converted into a reading.

Operating Characteristics of MPU-6050

- supply Voltage (V LOGIC): 1.8 V to 3.3 V (logic side).
- Operating Temperature Range: –40 °C to +85 °C.
- Accelerometer Full-Scale Ranges: ±2 g, ±4 g, ±8 g, ±16 g.
- Gyroscope Full-Scale Ranges: ±250°/s, ±500°/s, ±1000°/s, ±2000°/s.
- Interface Protocol: I2C.
- Internal ADC Resolution: 16-bit (for each axis)
- Built-in Temperature Sensor: Range –40 to +85°C

Fig. 3. Conductive Copper Tape.

3) Conductive Copper Tape: It acts as a capacitive safety interlock. By utilizing the ESP32-S3's built-in touch sensing, we ensure the car only moves when the user intentionally "pinches" their fingers, preventing accidental movement. One piece is placed on the thumb and another on the index finger. One is connected to a Touch GPIO and the other to Ground.

Fig. 4. 3.7V Li-Po Battery.

4) 3.7V Li-Po Battery: Portable Power Supply. Provides the necessary voltage and current for the S3 and MPU-6050 in a wearable, lightweight form factor.

5) Glove: A sports glove or work glove are used to attach the sensors in it and then wear it on the hand and hence there is no use of the breadboard.

B. The Receiver Node (The Vehicle)

1) ESP32-S3: Vehicle Controller. It listens for ESP-NOW packets from the glove. It then translates these tilt angles into PWM (Pulse Width Modulation) signals to control the speed and direction of the motors. Using the same chip as the transmitter ensures perfect protocol compatibility and allows the car to handle the Ultrasonic Sensor calculations on its second core.

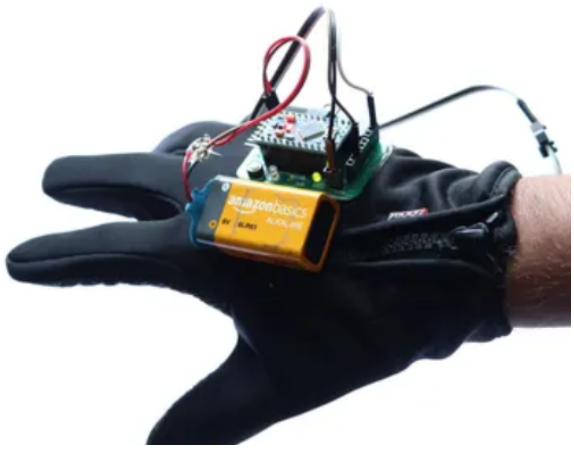

Fig. 5. Wearable glove.

2) HC-SR04 Ultrasonic Sensor: It emits an ultrasonic pulse and measures the time it takes to echo back. The S3 uses this time to calculate the distance to the nearest wall. This provides numerical distance data. If the distance is less than a predefined value, the S3 can "override" the user's hand command and stop the car to prevent a crash.

HC-SR04 Ultrasonic (US) sensor is a 4 pin module, whose pin names are Vcc, Trigger, Echo and Ground respectively. The module has two eyes like projects in the front which forms the Ultrasonic transmitter and Receiver. The sensor works with the simple formula that is: $Distance = Speed * Time$

The Ultrasonic transmitter transmits an ultrasonic wave, this wave travels in air and when it gets objected by any material it gets reflected back toward the sensor this reflected wave is observed by the Ultrasonic receiver module as shown in the fig 6

Now, to calculate the distance using the above formulae, we should know the Speed and time. Since we are using the Ultrasonic wave we know the universal speed of ultrasonic wave at room conditions which is 330m/s. The circuitry inbuilt on the module will calculate the time taken for the Ultrasonic wave to come back and turns on the echo pin high for that same particular amount of time, this way we can also know the time taken. Now simply calculate the distance using a microcontroller or microprocessor.

The TRIG transmits is set HIGH for 10microseconds, the sensor's internal control circuit detects that rising and falling edge for this detection to happen the TRIG pin must be on fot 10microseconds. After this, the control circuit automatically generates the ultrasonic burst using its own built-in oscillator and driver.i.e. The HC-SR04's transmitter emits 8 ultrasonic bursts at 40 kHz. As soon as the 8 pulses are finished, the module immediately starts its echo detection phase. Right at this moment, the ECHO pin is set HIGH. It remains HIGH for as long as the sensor is "waiting" for the echo to come

back. Basically, it calculates the flight time of the wave transmitted[2]

Fig. 6. Pinout Diagram of HC-SR04.

TABLE I
IMPORTANT CHARACTERISTICS OF HC-SR04

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2m
Measuring Angle	15 degree
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

Fig. 7. DC motors.

3) DC Motors and Chassis: The Actuators. Converts electrical energy into mechanical rotation to move the wheels. Gear motors provide high torque, allowing the robot to move smoothly on different surfaces.

4) Li-ion Battery Pack: Primary Vehicle Power and Provides high-density power for the motors. A Buck Converter or Voltage Regulator must be used to step this down to 5V to safely power the ESP32-S3.

Fig. 8. Li-ion battery.

5) L298N Motor Driver Module: The ESP32-S3 cannot provide the high current (Amps) needed to turn motors. The L298N takes a low-power signal from the S3 and switches a high-power battery source to the motors. It allows for bi-directional control (Forward/Reverse) and speed control via PWM.

DC motors are the easiest motors to use! If you connect a battery to a DC motor, it will spin in one direction. If you swap the wires, it spins in the other direction. But you can't always physically swap the wires every time you want to change direction. That's where an H-bridge comes in handy!

An H-bridge is a special circuit with four electronic switches arranged in an "H" shape, with the motor in the middle.

By turning these switches on and off in a specific order, you can make electricity flow through the motor in one direction or the opposite direction. This clever trick lets you control which way the motor spins without having to physically reverse any wires.

At its core, the L298N has two separate H-bridge circuits. Since the L298N has two of them, it can control two different DC motors at the same time.[3]

The L298N module has 11 pins in total.

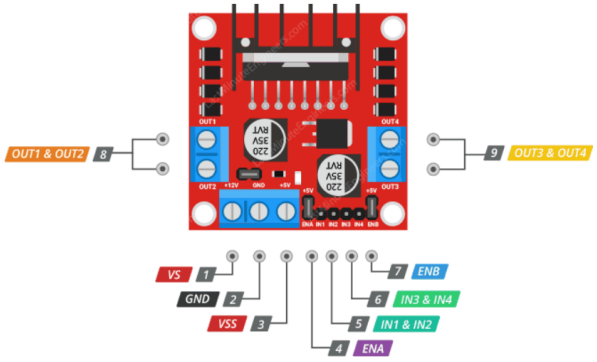

Fig. 9. Pinout descriotion of L298N Module.

- VS: is the main power input for the motors. On many modules, this pin is labeled as +12V, but it can actually handle a wide range of DC voltages, from 5V up to 46V. However, it's important to understand that the voltage your motors actually receive will be a bit lower than what

you supply to the VS pin. This is because the internal H-Bridge transistors inside the L298N cause a voltage drop of about 2V.

- VSS: is the power input for the internal logic circuitry of the chip. This logic circuitry needs a steady 5V to operate. You can supply this 5V in one of two ways. One option is to connect an external 5V power supply directly to the VSS pin. The other option is to use the onboard 5V voltage regulator, which takes power from the motor supply (VS) and automatically provides 5V to the logic circuits. When you use the onboard regulator, you don't need to connect anything to the VSS pin yourself.
- GND pin is the common ground connection for the module.
- The OUT1 and OUT2 pins connect to your first motor (motor A), while the OUT3 and OUT4 pins connect to your second motor (motor B). You can connect any DC motor that runs on 5 to 46 volts to these pins.
- The chip has two direction control pins for each motor. The IN1 and IN2 pins control the spinning direction of motor A, while the IN3 and IN4 pins control the spinning direction of motor B.

  By setting different combinations of HIGH or LOW signals on these pins, you can make the motors spin forward, backward, or stop. The chart below shows exactly how this works:[3]

TABLE II
DIRECTION CONTROL PINS

Input1	Input2	Spinning Direction
Low(0)	Low(0)	Motor OFF
High(1)	Low(0)	Forward
Low(0)	High(1)	Backward
High(1)	High(1)	Motor OFF

- The ENA and ENB pins control how fast your motors spin. Actually, these pins work as simple on/off switches. When you pull one of them HIGH, the corresponding motor is enabled and spins at full speed. When you pull it LOW, the motor is completely disabled and won't spin at all.

  But these pins can do more than just switch the motors on or off. By sending a PWM (Pulse Width Modulation) signal to ENA or ENB, you can actually control the speed of each motor.

  The L298N module usually comes with jumpers placed across the ENA and ENB pins, which connects them directly to 5V. This makes the motors spin at full speed by default. If you want to control the speed of your motors through your MCU, you'll need to remove these jumpers and connect ENA and ENB pins to the PWM-enabled pins on your MCU.[3]

Features and Specifications

- Driver Model: L298N 2A
- Driver Chip: Double H Bridge L298N
- Motor Supply Voltage (Maximum): 46V
- Motor Supply Current (Maximum): 2A
- Logic Voltage: 5V
- Driver Voltage: 5-35V
- Driver Current:2A
- Logical Current:0-36mA
- Maximum Power (W): 25W
- Current Sense for each motor
- Heatsink for better performance
- Power-On LED indicator

Note: The relay module was removed from the final system architecture to optimize the power-to-weight ratio and improve the overall efficiency of the mobile platform. In a wearable-controlled robotic system, the significant quiescent current draw required to maintain a mechanical relay's electromagnetic coil in an "on" state creates an unnecessary drain on the battery without providing a proportional increase in functional utility. Furthermore, by transitioning from mechanical switching to Software-Defined Safety (SDS), the design eliminates mechanical switching latency and contact arcing. The system instead leverages the ESP32-S3's real-time processing capabilities to implement high-speed electronic braking via the L298N motor driver, using distance telemetry from the HC-SR04 ultrasonic sensor to provide a more responsive, proportional, and energy-efficient safety intervention than a binary hardware disconnect.

IV. COMMUNICATION: ESP-NOW PROTOCOL

ESP-NOW is a connectionless communication protocol developed by Espressif that allows multiple devices to talk to each other without using Wi-Fi. It is a Layer 2 (Data Link Layer) protocol, meaning it sends data packets directly from one MAC address to another.

If you use Wi-Fi, the ESP32 has to manage the heavy overhead of the TCP/IP stack (assigning IP addresses, checking packets). If the signal drops, the car stops responding while it tries to "reconnect." With ESP-NOW, the glove "blasts" the tilt data and the car "hears" it instantly. If a packet is lost, the next one arrives 5ms later without any reconnection delay.

The communication link between the transmitter (glove) and receiver (vehicle) utilizes the ESP-NOW protocol. Unlike standard Wi-Fi (IEEE 802.11 b/g/n), which requires a persistent connection to an Access Point and introduces significant latency through the TCP/IP stack, ESP-NOW operates at the Data Link Layer. This connectionless approach enables a deterministic, low-latency response, which is critical for real-time robotic tele-operation. This protocol utilizes vendor-specific action frames, allowing for high-speed transmission with minimal power consumption, making it ideal for the battery-constrained wearable node.[4]

The Operational Workflow of ESP-NOW

The working of ESP-NOW can be broken down into four distinct stages:

- MAC Address Identification: Every ESP32-S3 has a unique, permanent hardware ID called a MAC Address. Before communication starts, the Transmitter is told the MAC Address of the Receiver
- Pairing (Software Level): Even though there is no "connection," the devices are "paired" in the code. This means they agree on a specific Wi-Fi Channel (usually channel 1) so they are listening on the same frequency.

- Packet Construction: The ESP32-S3 takes your data (e.g., $4X = 45, Y = -10, Button = 1$) and wraps it in a Vendor-Specific Action Frame.5 This is a small, lightweight envelope that doesn't have the "weight" of a standard internet packet.
- Direct Broadcast: The Transmitter "blasts" the packet into the air. The Receiver, which is constantly scanning that specific channel for that specific MAC Address, catches the packet, verifies it, and processes the command instantly.

REFERENCES

[1] Circuit Digest, "Esp32 air mouse using hand gesture control," *Circuit Digest - Electronics Projects*, 2023, accessed: Jan. 10, 2026. [Online]. Available: https://circuitdigest.com/microcontroller-projects/esp32-air-mouse-using-hand-gesture-control

[2] Components101, "Hc-sr04 ultrasonic sensor pinout, features, and datasheet," *Components101 Sensor Reference*, 2022, accessed: Jan. 10, 2026. [Online]. Available: https://components101.com/sensors/ultrasonic-sensor-working-pinout-datasheet

[3] Last Minute Engineers, "In-depth: Interface l298n dc motor driver module with arduino," *Last Minute Engineers Tutorials*, 2023, accessed: Jan. 10, 2026. [Online]. Available: https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/

[4] Espressif Systems, "ESP-NOW technical documentation - ESP-IDF programming guide," *Espressif Systems Technical Reference*, 2024, accessed: Jan. 10, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/network/esp_now.html